

Teaching Statistical Computing in the Age of AI

ICOTS 12 - Brisbane

Colin Rundel

Duke University

Who am I

Assoc. Professor of the Practice, Dept. of Statistical Science, Duke University

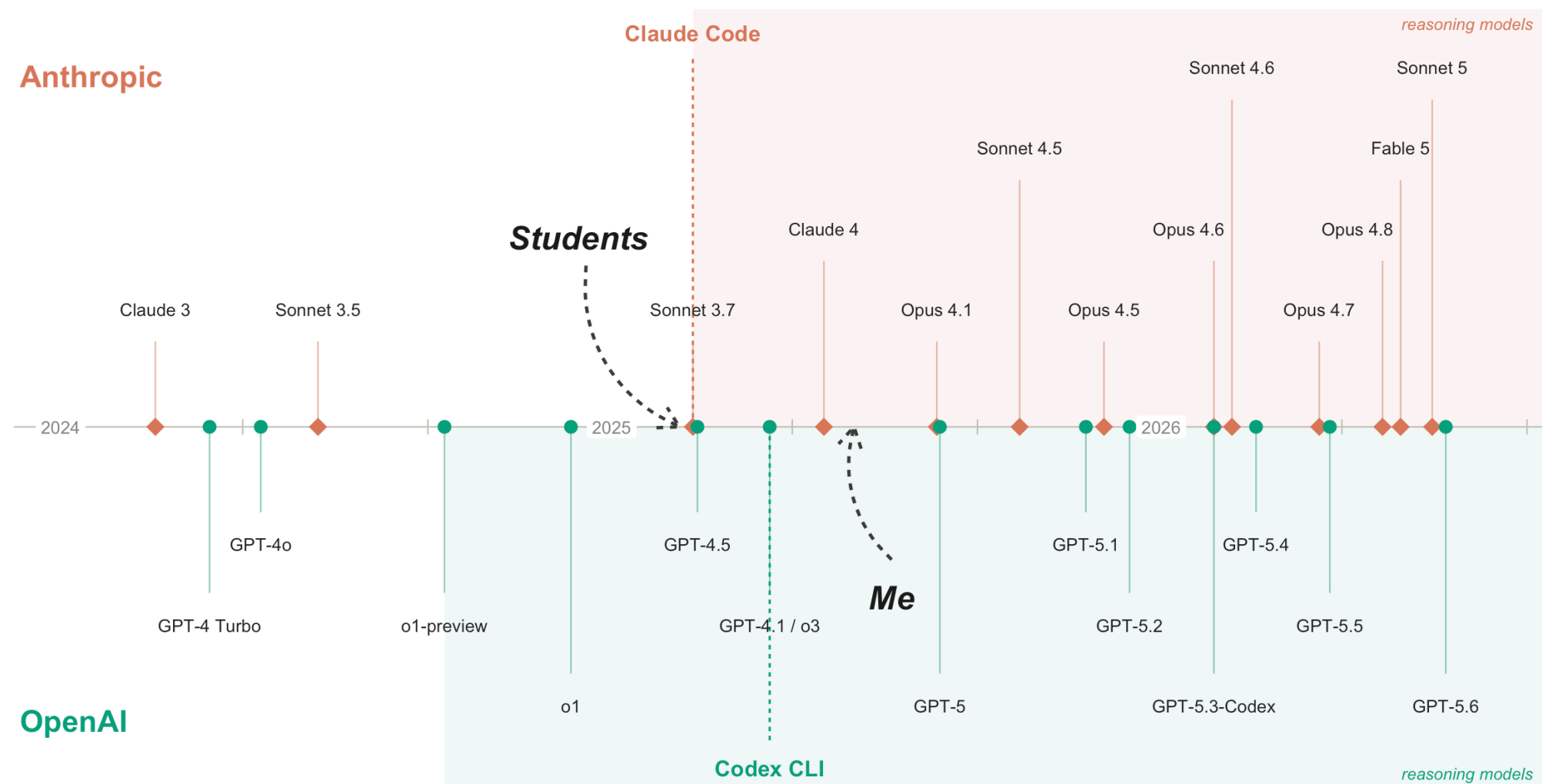
- [Sta 523](#) - Programming for Statistical Science  *Fall 2014*
R, 1st year MS required
- [Sta 323](#) - Statistical Computing  *Spring 2016*
R, 2nd/3rd year UG elective / required since 2023
- [Sta 663](#) - Statistical Computing and Computation  *Spring 2022*
Python, 1st year MS required

Learning objectives

Mastery of:

- Programming in R / Python
- Data wrangling
- Visualization
- Text data
- Modeling frameworks
- Version control
- Web scraping & APIs
- Interactive tools
- Databases & SQL
- Optimization

An AI Timeline



Syllabus (Fall 2024 - now)

Sharing / reusing code policy

- We are aware that a huge volume of code is available on the web, and many tasks may have solutions posted.
- Unless explicitly stated otherwise, this course's policy is that you may make use of any online resources (e.g. Google, StackOverflow, etc.) but you must explicitly cite where you obtained any code you directly use or use as inspiration in your solution(s).
- Any recycled code that is discovered and is not explicitly cited will be treated as plagiarism, regardless of source.
- *The same applies to the use of LLMs like ChatGPT, Claude, or GitHub Copilot - you are welcome to make use of these tools as the basis for your solutions but you must cite the tool when using it for significant amounts of code generation.*

Why do I care?

- By Spring 2025 starting to see >50% usage
- Usage starting to lean towards copy / paste (better models)
- Compression in the lower tail of the grade distribution
 - AI one-shot solutions going from 20-40% → 60% → 80% grade
 - Feedback focusing more on minor details than the big picture
- Declining lecture, lab, and OH attendance
- Arguing about “well that’s what AI told me to do”

Why don't I care?

- Writing code was never the goal of the course - code is a means to an end
 - Statistical and computational thinking for solving problems
 - Opinionated survey of the ecosystem
- Students were not writing most of their code anyway (team based assignments)
- Writing code vs reading code
- Students will be using these tools in the future (academics or industry)

Low-hanging fruit

Adjusting assessment - moving to in-class

- Random in-class quizzes - 5 MC questions in 5 min
 - Dramatically improves attendance
 - Forces students to stay current with material
- In-class exams
 - No writing actual code
 - Code reading and interpretation
 - Pseudo-code solution sketches

A more radical revision

For the 2026/2027 running of the MSS courses (Sta 523 & 663) we will be trying something:

- Combining the majority of the existing materials of *both* courses into the Fall semester exclusively
- Teaching R & Python concurrently,
 - focusing on the top-down concepts rather than the bottom-up syntax
- Provide / expect use of agentic tools from day 1
 - Who is going to pay is TBD (more on this in a bit)

Why?

- MSS students are coming in with strong(er) computing backgrounds
- Already using AI, but doing it badly / inconsistently in general
- Using these tools effectively probably should be a learning objective of a modern computing curriculum
- Address a demand for Python in the 1st semester (summer internships)
- Frees up 3-5 weeks of the 2nd course for an extended project-based assignment
 - Work on ambitious open-ended student-driven projects
 - Documentation and reporting on process and workflow, not just final results

Why not the UG course (Sta 323)?

- Single semester course taught by multiple faculty members
- Less confidence this approach will work for students with less computing experience
- Less pressure for Python in the curriculum

Aside - Who is going to pay?

This is a critical open question and the landscape is changing, currently two possibilities:

- Personal fixed cost plans
 - \$20/\$100/\$200 / month
 - 5 hr & weekly usage limits
- Pay for usage / pay per token plans
 - \$2-10 / 1M input tokens
 - \$15-50 / 1M output tokens

Enterprise licenses for these tools are trending very strongly towards the latter option exclusively.

Very real concerns about data privacy / FERPA / etc. if not working under the auspices of your university.

What does pay per token look like?

It is hard to directly compare, but expect costs to be 5-20x more than a fixed cost plan for moderate to heavy use.

My last 30 days of Claude Code usage (Jun 15 - Jul 13), priced at standard API rates:

Model	Input	Output	Cache reads	Cache writes	Cost
Fable 5	0.5M	3.1M	565M	18.8M	\$1,059
Opus 4.8	2.6M	4.9M	979M	43.0M	\$944
Haiku 4.5	~0	~0	6.1M	0.4M	\$1
Total					\$2,004

What are we going to do?

Duke has licensing agreements with OpenAI & Anthropic - both only offer pay per token options (or equivalent).

- Cost controls are very limited and not viable for use with students / teaching
- Working towards using a Claude Team plan for the course (~\$25 / seat / month)
- Alternative is to ask students to bring their own plan

AI on the instructors side

- Tools are great at ingesting your lectures and generating lots of mediocre multiple choice questions
- Frontier models are also really good at reviewing code
 - Limited support for anonymisation in [ghclass](#)'s grading workflow
 - Template and rubric automation in [markermd](#)
 - Value in assessing marking consistency
 - Repeated passes are a necessity

Reach out



rundel.github.io



[rundel](#)



rundel@gmail.com
colin.rundel@duke.edu



sta523-fa25.github.io
sta323-sp26.github.io
sta663-sp26.github.io



sta523-fa26.github.io